

JOBSHEET PRAKTIKUM PERTEMUAN 11

Materi : Store Procedure
Program Studi : D3 Teknik Informatika
Mata Kuliah : Sistem Basis Data
Semester : 2
Pertemuan : Stored Procedure

A. CAPAIAN PEMBELAJARAN

Mahasiswa mampu merancang dan mengimplementasikan Stored Procedure pada sistem basis data untuk mengotomatisasi pengolahan data, meningkatkan efisiensi eksekusi query, serta mendukung kebutuhan aplikasi berbasis database.

B. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini, mahasiswa mampu:

1. Memahami konsep Stored Procedure pada MySQL.
 2. Membuat Stored Procedure sederhana.
 3. Menjalankan Stored Procedure menggunakan perintah `CALL`.
 4. Membuat Stored Procedure dengan parameter.
 5. Menggunakan Stored Procedure untuk menampilkan, menambah, dan menghapus data.
 6. Menerapkan Stored Procedure dalam kasus sederhana pengelolaan data.
-

C. DASAR TEORI

1. Pengertian Stored Procedure

Stored Procedure adalah sekumpulan perintah SQL yang disimpan di dalam database dan dapat dijalankan kembali kapan saja tanpa perlu menuliskan perintah SQL tersebut secara berulang. Stored Procedure dapat diibaratkan sebagai program kecil yang berada di dalam database untuk melakukan tugas tertentu.

Pada saat mengelola database, sering kali pengguna menjalankan query yang sama berulang-ulang. Misalnya menampilkan data produk, mencari data berdasarkan kriteria tertentu, atau menambahkan data baru ke dalam tabel. Jika query tersebut selalu ditulis ulang, pekerjaan menjadi kurang efisien dan berpotensi menimbulkan kesalahan penulisan.

Sebagai contoh, untuk menampilkan seluruh data produk diperlukan query berikut:

```
SELECT * FROM produk;
```

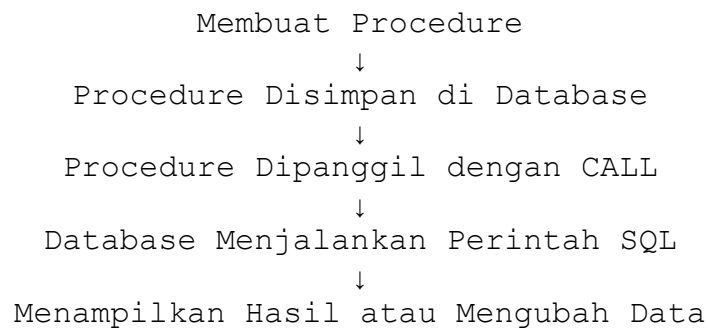
Jika query tersebut sering digunakan, maka lebih baik disimpan ke dalam Stored Procedure. Setelah procedure dibuat, pengguna cukup menjalankan:

```
CALL tampil_produk();
```

Database akan mengeksekusi seluruh perintah yang telah disimpan dalam procedure tersebut.

2. Cara Kerja Stored Procedure

Secara sederhana, alur kerja Stored Procedure adalah sebagai berikut:



Stored Procedure disimpan sebagai objek database sehingga dapat digunakan berkali-kali tanpa perlu membuat ulang query yang sama.

3. Keuntungan Menggunakan Stored Procedure

a. Mengurangi Penulisan Query Berulang

Query yang sering digunakan cukup dibuat satu kali dan dapat dipanggil kembali kapan saja.

b. Mempermudah Pemeliharaan Sistem

Jika terdapat perubahan pada proses pengolahan data, pengembang cukup mengubah isi procedure tanpa perlu mengubah kode pada aplikasi.

c. Meningkatkan Keamanan

Administrator dapat memberikan hak akses kepada pengguna untuk menjalankan procedure tanpa memberikan akses langsung ke tabel.

d. Meningkatkan Efisiensi

Karena procedure sudah tersimpan pada server database, proses eksekusi dapat lebih cepat dibandingkan mengirim query yang sama berulang kali dari aplikasi.

4. Struktur Dasar Stored Procedure

Sintaks dasar pembuatan Stored Procedure:

```
DELIMITER $$

CREATE PROCEDURE nama_procedure()
BEGIN
    -- Perintah SQL
END $$

DELIMITER ;
```

Keterangan:

Perintah	Fungsi
CREATE PROCEDURE	Membuat procedure baru
BEGIN	Awal blok perintah
END	Akhir blok perintah
CALL	Menjalankan procedure
DELIMITER	Mengubah tanda akhir perintah sementara

5. Parameter pada Stored Procedure

Parameter digunakan agar procedure dapat menerima nilai dari pengguna saat dijalankan.

Contoh:

```
CALL cari_produk('Laptop');
```

Nilai "Laptop" akan digunakan sebagai parameter pencarian.

Jenis parameter yang sering digunakan adalah:

IN

Digunakan untuk menerima nilai dari pengguna.

OUT

Digunakan untuk mengembalikan nilai dari procedure.

INOUT

Digunakan sebagai parameter masukan sekaligus keluaran.

6. Perbedaan Stored Procedure dan Trigger

Stored Procedure	Trigger
Dijalankan manual menggunakan CALL	Berjalan otomatis
Dapat dipanggil kapan saja	Dipicu oleh INSERT, UPDATE, DELETE
Dapat menerima parameter	Tidak menerima parameter saat dijalankan
Digunakan untuk proses tertentu	Digunakan untuk proses otomatis

7. Penerapan Stored Procedure

Stored Procedure banyak digunakan pada:

- Sistem Informasi Akademik
- Sistem Rumah Sakit
- Sistem Penjualan
- Sistem Kepegawaian
- Sistem Perbankan

Karena mampu membuat pengelolaan database menjadi lebih terstruktur, aman, dan efisien.

D. ALAT DAN BAHAN

1. Laptop/Komputer
 2. XAMPP atau Laragon
 3. MySQL/MariaDB
 4. phpMyAdmin (Opsional)
-

E. LANGKAH PRAKTIKUM

1. Membuat Database

```
CREATE DATABASE db_toko;  
USE db_toko;
```

2. Membuat Tabel Produk

```
CREATE TABLE produk (  
    id_produk INT AUTO_INCREMENT PRIMARY KEY,
```

```
nama_produk VARCHAR(100),  
harga INT,  
stok INT  
);
```

3. Menambahkan Data Awal

```
INSERT INTO produk(nama_produk,harga,stok)  
VALUES  
( 'Laptop',8000000,10),  
( 'Mouse',150000,25),  
( 'Keyboard',250000,15),  
( 'Monitor',2500000,8);
```

F. PRAKTIKUM PARAMETER IN

Membuat Procedure Menampilkan Data Produk

```
DELIMITER $$  
  
CREATE PROCEDURE tampil_produk()  
  
BEGIN  
  
    SELECT * FROM produk;  
  
END $$  
  
DELIMITER ;
```

Menjalankan:

```
CALL tampil_produk();
```

Membuat Procedure dengan Parameter IN

```
DELIMITER $$  
  
CREATE PROCEDURE cari_produk(  
    IN p_nama VARCHAR(100)  
)  
BEGIN  
    SELECT *  
    FROM produk  
    WHERE nama_produk = p_nama;  
END $$
```

```
DELIMITER ;
```

Menjalankan:

```
CALL cari_produk('Laptop');
```

Membuat Procedure Menambah Data

```
DELIMITER $$
```

```
CREATE PROCEDURE tambah_produk(  
    IN p_nama VARCHAR(100),  
    IN p_harga INT,  
    IN p_stok INT  
)  
BEGIN  
    INSERT INTO produk(nama_produk,harga,stok)  
    VALUES(p_nama,p_harga,p_stok);  
END $$
```

```
DELIMITER ;
```

Menjalankan:

```
CALL tambah_produk('Printer',1200000,5);
```

G. PRAKTIKUM PARAMETER OUT

Menghitung Jumlah Produk

```
DELIMITER $$
```

```
CREATE PROCEDURE hitung_jumlah_produk(  
    OUT total_produk INT  
)  
BEGIN  
    SELECT COUNT(*)  
    INTO total_produk  
    FROM produk;  
END $$
```

```
DELIMITER ;
```

Menjalankan:

```
CALL hitung_jumlah_produk(@jumlah);
```

Menampilkan hasil:

```
SELECT @jumlah;
```

Mengambil Harga Produk Berdasarkan ID

```
DELIMITER $$

CREATE PROCEDURE ambil_harga_produk(
    IN p_id INT,
    OUT p_harga INT
)
BEGIN
    SELECT harga
    INTO p_harga
    FROM produk
    WHERE id_produk = p_id;
END $$

DELIMITER ;
```

Menjalankan:

```
CALL ambil_harga_produk(1,@harga_produk);

SELECT @harga_produk;
```

H. PRAKTIKUM PARAMETER INOUT

Menambah Nilai Variabel

```
DELIMITER $$

CREATE PROCEDURE tambah_lima(
    INOUT angka INT
)
BEGIN
    SET angka = angka + 5;
END $$

DELIMITER ;
```

Menjalankan:

```
SET @nilai = 10;

CALL tambah_lima(@nilai);
```

```
SELECT @nilai;
```

Hasil:

15

I. PRAKTIKUM UPDATE DAN DELETE

Update Stok Produk

```
DELIMITER $$

CREATE PROCEDURE update_stok(
    IN p_id INT,
    IN p_stok INT
)
BEGIN
    UPDATE produk
    SET stok = p_stok
    WHERE id_produk = p_id;
END $$

DELIMITER ;
```

Menjalankan:

```
CALL update_stok(1,20);
```

Hapus Produk

```
DELIMITER $$

CREATE PROCEDURE hapus_produk(
    IN p_id INT
)
BEGIN
    DELETE FROM produk
    WHERE id_produk = p_id;
END $$

DELIMITER ;
```

Menjalankan:

```
CALL hapus_produk(2);
```

E. PRAKTIKUM

Kerjakan latihan berikut secara mandiri.

Latihan 1

Buat Procedure bernama `tampil_semua_produk` yang menampilkan seluruh data pada tabel produk.

Latihan 2

Buat Procedure bernama `produk_mahal` yang menampilkan produk dengan harga lebih dari Rp1.000.000.

Latihan 3

Buat Procedure bernama `produk_stok_sedikit` yang menampilkan produk dengan stok kurang dari 10.

Latihan 4

Buat Procedure bernama `cari_stok` yang menerima parameter stok dan menampilkan produk sesuai stok yang dicari.

Contoh:

```
CALL cari_stok(15);
```

Latihan 5

Buat Procedure bernama `cari_harga` yang menerima parameter harga dan menampilkan produk dengan harga lebih besar dari nilai yang diberikan.

Contoh:

```
CALL cari_harga(500000);
```

Latihan 6

Buat Procedure bernama `tambah_barang` untuk menambahkan data produk baru menggunakan parameter.

Uji dengan data berikut:

Nama Produk	Harga	Stok
Flashdisk	120000	30

Latihan 7

Buat Procedure bernama `update_stok` yang digunakan untuk mengubah stok produk berdasarkan ID Produk.

Contoh:

```
CALL update_stok(2, 50);
```

Latihan 8

Buat Procedure bernama `hapus_produk_nama` yang menghapus produk berdasarkan nama produk.

Contoh:

```
CALL hapus_produk_nama('Mouse');
```

Latihan 9

Buat Procedure bernama `jumlah_produk` yang menampilkan jumlah seluruh data produk menggunakan fungsi `COUNT()`.

Latihan 10 (Studi Kasus)

Buat sebuah Stored Procedure bernama `produk_tersedia` yang hanya menampilkan produk dengan stok lebih dari 0.

8. Lembar Hasil / Laporan Praktikum

Laporan dalam format PDF berisi:

1. Script SQL
2. Screenshot hasil query.
3. Analisis Praktikum:
 1. Apa yang dimaksud dengan Stored Procedure?
 2. Jelaskan fungsi perintah `CALL`.
 3. Apa fungsi parameter `IN`?

4. Apa fungsi parameter OUT?
5. Apa fungsi parameter INOUT?
6. Jelaskan perbedaan Stored Procedure dan Trigger.
7. Mengapa Stored Procedure dapat meningkatkan efisiensi database?
8. Apa keuntungan penggunaan Stored Procedure dalam aplikasi?
9. Kapan parameter OUT lebih tepat digunakan dibandingkan parameter IN?